

InfraFlow

Five days from blank Terraform to deployed MVP

A real project story, reconstructed from the codebase, docs, and git history.

Visual AWS architecture. Terraform generation.

Real deployment execution. Storage lessons.

GitHub Actions runners. Safety gates.

Working product loop.

BUILT WITH REAL AWS

Canvas

Terraform

GitHub Actions

AWS

InfraFlow

Five days from blank Terraform to deployed MVP

This PDF tells the real story of InfraFlow: a visual AWS infrastructure builder that moved from a simple Terraform idea into a working product with a React dashboard, Node.js backend, Terraform generation, AWS deployment, GitHub Actions execution, application pipelines, notifications, support workflows, and admin controls.

The story is grounded in the actual repo: docs/HLD.md, docs/LLD.md, IAAS backend/STORAGE.md, the current frontend and backend source, and the Git history. External pricing context is based on current public pages for HashiCorp HCP Terraform, Spacelift, and env0.

Clickable table of contents

- [Investigation Notes](#)
- [The Starting Point](#)
- [The Build](#)
- [By The Numbers](#)
- [What Works Today](#)
- [The Architecture](#)
- [The Hard Problems](#)
- [Technical Concepts Learned](#)
- [Impact and Market Context](#)
- [Future Potential](#)
- [The Five Days](#)
- [Closing](#)

Investigation Notes

The first pass through the codebase showed that InfraFlow is not just a diagramming UI. It has a full product skeleton:

- A Vite React frontend with landing page, authentication, dashboard, visual builder, deployment page, application pipeline builder, AI agent page, support tickets, and super admin console.
- A Node.js Express backend with MongoDB/Mongoose models for users, workspaces, AWS accounts, diagrams, deployments, notifications, application pipelines, tickets, and audit logs.
- AWS account connection through role-based access, AWS live sync, cost and resource insight surfaces, and deployment execution against real AWS.
- Terraform generation for supported AWS node types, including Lambda, API Gateway, S3, DynamoDB, SQS, SNS, EventBridge, IAM, VPC, EC2, CloudWatch, Secrets Manager, CloudFront, WAF, and others.
- Two Terraform execution paths: a local backend runner and a GitHub Actions runner selected by `Deployment.executor`.
- Storage abstraction through `StorageAdapter`, `LocalStorageAdapter`, and `S3StorageAdapter`, with Lambda zip metadata and S3 artifact references.
- Safety logic centralized in `deploymentGuards.js` so local and GitHub runners share the same failure cleanup rules.

Important caveat: the repo's first commit is dated 2026-06-22, while the intense GitHub pipeline and deployment activity appears across 2026-07-21 through 2026-07-28. The "five days" narrative is therefore treated as the intense MVP build sprint, not the entire lifetime of the repository.

The Starting Point

InfraFlow started from a practical pain: infrastructure-as-code is powerful, but writing HCL by hand is slow, easy to misconfigure, and hard to explain visually. The initial goal was simple:

Aha Draw AWS infrastructure like a workflow, generate Terraform, then deploy it.

That idea forced the first architectural decision: InfraFlow could not remain a mock canvas. It needed a real path from a diagram node to a Terraform resource, from Terraform to AWS, and from AWS back into the dashboard as status, logs, and outputs.

The first meaningful milestone was the classic serverless backbone:

- Lambda function
- IAM execution role
- API Gateway HTTP API
- Terraform runner
- Deployment record and logs

Once that existed, every later feature had a test: can this visual action survive the path from browser to backend to Terraform to AWS?

The Build

1. The canvas became a compiler

The visual builder uses React Flow to let a user place AWS services on a diagram, connect them, edit properties, validate the topology, and export Terraform. That sounds visual, but the real engineering work is compiler-like:

- A node has service identity, config, region, labels, and ports.
- An edge has relationship semantics.
- A deployment planner counts resources, relationships, warnings, blockers, and regions.
- A Terraform generator turns the graph into HCL.
- Unsupported or incomplete resources are surfaced as validation issues instead of silently ignored.

This made the builder more than a drawing board. It became a structured infrastructure authoring surface.

2. The first real deploy path exposed real AWS friction

The backend deployment flow creates a diagram record, validates it, generates Terraform, stores a deployment record, writes `main.tf`, stages Lambda artifacts, runs `terraform init`, gates on `terraform plan`, then applies.

The first hard lessons were not theoretical:

- IAM role ARNs are not IAM user ARNs.
- Terraform needs read permissions after creating resources, not just create permissions.
- Lambda handler names and zip roots must match the runtime's import expectations.
- API Gateway, Lambda permissions, and CORS must be configured as one operational chain.

The work moved from "generate HCL" to "generate HCL that survives the AWS provider's full create-read-refresh lifecycle."

3. The Lambda zip bug revealed the execution model

The Lambda node originally exposed a "Deployment zip path" field. That is fine if Terraform runs on the same machine as the browser. But InfraFlow's Terraform runner is backend-side, and later GitHub-side. A local browser path is meaningless there.

Aha Aha: this was not a file picker bug. It was an execution boundary bug.

The fix was architectural: upload the zip to the backend, store it through the storage adapter, keep metadata beside it, and let Terraform reference something the runner can actually access. In S3 mode, the Lambda package is referenced by `s3_bucket` and `s3_key`; the source hash is supplied from upload-time metadata.

4. The storage problem became a product architecture problem

Terraform runs generate more than a single file. They create working directories, provider caches, plans, state, uploaded artifacts, and metadata. The project hit a storage bottleneck that grew to roughly 6 GB. The identified causes were:

- Terraform provider/plugin cache accumulation.
- Per-deployment working directories.
- State and plan files.
- Uploaded Lambda artifacts and metadata.

The fix was not just "delete files." InfraFlow introduced a storage boundary:

- `StorageAdapter` as the durable artifact interface.
- `LocalStorageAdapter` for local development.
- `S3StorageAdapter` for durable cloud artifact storage.
- Ephemeral local working directories for Terraform CLI execution.
- Cleanup logic for provider caches and orphaned Lambda zip uploads.

That distinction matters: Terraform still needs a local scratch directory to run, but artifacts that must outlive a run belong in durable storage.

5. Terraform execution moved to GitHub Actions

The next scaling pressure was compute. A backend process running Terraform is simple, but it creates operational coupling:

- Long-running applies tie up backend resources.
- Provider downloads grow disk use.
- Process restarts can orphan state.
- Horizontal scaling becomes harder.

InfraFlow added a `github-actions` executor. The backend pushes generated Terraform into a platform repo, dispatches `terraform-deploy.yml`, polls the workflow run, and receives a callback with outputs and state status.

The key design choice was not to duplicate deployment semantics. Both local and GitHub runners share the same contract and route through the same dispatcher. Safety-critical failure logic lives in one place.

6. The validation pipeline became three layers

InfraFlow's deployment path now has layered gates:

- Design-time validation in the visual builder.
- Terraform syntax/provider validation through `terraform validate`.
- Terraform `plan` before `apply`, with failures reported as "nothing was changed in AWS."

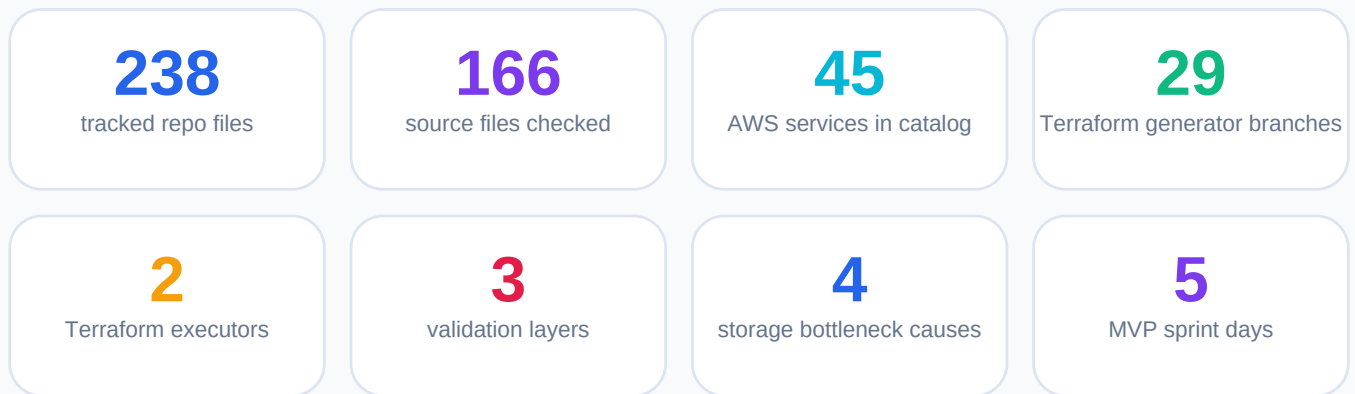
That makes deployment failure more explainable. A bad diagram should fail before Terraform. Bad HCL should fail before planning. A provider or permission issue should fail at plan before AWS is changed whenever possible.

7. The product became deployable

The frontend was prepared for CloudFront/S3 hosting. The backend was adapted for Lambda/API Gateway, including CORS behavior and Lambda-safe environment loading. MongoDB moved from local assumptions to Atlas-ready configuration. API base URLs were centralized so the frontend does not accidentally call itself in production.

By the end of the sprint, InfraFlow could design infrastructure, generate Terraform, run deployments, watch logs, show outputs, and trigger app delivery through GitHub.

By The Numbers



- 238 tracked files in the repository.
- 166 frontend and backend source files under `src` and `IAAS backend/src`.
- 45 AWS services listed in the visual builder catalog.
- 29 Terraform generator branches in the backend generator.
- 2 Terraform execution modes: local backend runner and GitHub Actions runner.
- 3 deployment validation layers: diagram validation, Terraform validate, Terraform plan gate.
- 4 storage bottleneck causes identified: provider cache, work directories, state/plan files, uploaded Lambda artifacts.
- 3 storage concepts separated: durable artifacts, ephemeral Terraform workdirs, generated Terraform state/backend configuration.
- 3 public competitor pricing anchors checked: HCP Terraform, Spacelift, env0.

What Works Today

InfraFlow's current MVP has a broad working surface:

- Public SaaS landing page.
- Login and registration.
- Dashboard shell with AWS account state and navigation.
- AWS account connection and live sync.
- Visual builder with drag/drop AWS services, edges, properties, grouping, validation, and export.
- Terraform generation and deployment records.
- Local Terraform executor.
- GitHub Actions Terraform executor.
- Lambda zip upload handling.
- Deployment logs, notifications, resource info, update, destroy, force destroy, and merge-source safeguards.
- Application pipeline builder that syncs generated GitHub Actions files into a selected repo.
- GitHub OAuth, repository and branch selection, workflow dispatch, and run status polling.
- AI agent page, support tickets, and super admin console.

This is not yet a mature Terraform Cloud replacement. It is a working MVP with the core loop in place: design, validate, generate, deploy, observe, and iterate.

The Architecture

Frontend

The frontend is a Vite React application. The most important product areas are:

- `src/components`: builder canvas, toolbar, sidebar, properties panel, deployment modal.
- `src/dashboard`: dashboard shell, AWS insights, app pipelines, support, super admin, resource info.
- `src/utils`: Terraform export, deployment API, validation, resource requirements.
- `src/store`: diagram state and validation flow.

The visual builder is the user's primary interface. It converts infrastructure intent into typed nodes and edges, then presents validation and deployment actions in the same flow.

Backend

The backend is a Node.js Express API. Key areas:

- `controllers`: auth, AWS accounts, deployments, GitHub, app pipelines, tickets, admin.
- `models`: users, workspaces, diagrams, deployments, notifications, app pipelines, tickets.
- `services`: AWS role credentials, Terraform runners, GitHub Actions client, storage-backed Lambda uploads, OIDC role provisioning.
- `utils`: Terraform generator, deployment planner, validators, audit helpers.

MongoDB stores the product state: users, workspaces, diagrams, deployments, logs, tickets, notifications, and pipeline records.

Execution

The local executor writes `main.tf`, stages artifacts, runs Terraform commands in a local workdir, captures outputs, and updates deployment status.

The GitHub Actions executor pushes generated Terraform to a GitHub repo, dispatches a workflow, polls run status, receives callback output, and finalizes the same deployment record.

Both execution paths share the same safety rules. That is the important invariant.

The Hard Problems

IAM trust vs permissions

GitHub Actions OIDC deployment required separating two policies:

- Trust policy: who is allowed to assume the role.
- Permissions policy: what the assumed role is allowed to do.

This distinction appears in the generated `deploy/oidc-trust-policy.json` and `deploy/oidc-permissions-policy.json` files for application pipelines.

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringLike": {
      "token.actions.githubusercontent.com:sub": "repo:OWNER/REPO:ref:refs/heads/main"
    }
  }
}
```

Local file vs remote runner

The Lambda zip path issue was the clearest architectural bug. A file path selected in the browser is not available to:

- the backend server,
- a future worker,
- or a GitHub-hosted Terraform runner.

The solution was to upload the artifact and store it through a backend-controlled abstraction.

```
class StorageAdapter {
  async writeFile(key, content) {}
  async readFile(key) {}
  async deleteFile(key) {}
  async exists(key) {}
  async getWorkingDirectory(deploymentId) {}
  resolveArtifactReference(key) {}
}
```

Plan before apply

The runner now treats Terraform plan as a safety gate. If plan fails, the error says nothing was changed in AWS. That is a small product detail with real operational value.

CORS and CloudFront/API Gateway

The production frontend uses CloudFront. The backend sits behind API Gateway/Lambda. That means browser calls need the right production API base URL and API Gateway/Lambda must return correct CORS headers.

The final shape is one product experience across two delivery paths:

```
Browser
-> CloudFront frontend
-> API Gateway /api/v1
-> Lambda Express backend
-> MongoDB Atlas
```

Technical Concepts Learned

Terraform state management

State is not just a file. It is the memory of deployed infrastructure. Losing it or placing it in the wrong execution boundary breaks updates, destroys, and drift checks.

Bottleneck vs resource leak

The 6 GB storage issue was a bottleneck because normal usage accumulated heavy working data. Some files were expected; the problem was lifecycle ownership. The fix was not just deletion, but separating durable artifacts from ephemeral scratch.

Execution-model architecture

Moving Terraform execution to GitHub Actions changed where compute, disk, credentials, and logs live. That forced explicit artifact storage, callbacks, status polling, and restart reconciliation.

Safety-critical invariants

The most important invariant: never auto-destroy already-working infrastructure after a failed update. A failed first-time deploy can clean up newly created resources. A failed update must preserve what was already running.

CI/CD as execution layer

GitHub Actions is used in two ways:

- As application CI/CD for deploying frontend/container/Lambda code from user repos.
- As Terraform execution infrastructure for InfraFlow-generated Terraform.

Those are related patterns, but they solve different problems.

Impact and Market Context

InfraFlow targets engineers and teams who want visual-first Terraform generation and deployment without starting from hand-written HCL. It is positioned honestly:

- Terraform Cloud/HCP Terraform is a mature managed Terraform platform with remote operations, state, VCS integration, and governance. HashiCorp's public pricing currently lists HCP Terraform Essentials from \$0.10/resource/month, Standard from \$0.47/resource/month, and Premium from \$0.99/resource/month.
- Spacelift is a mature orchestration platform with Terraform/OpenTofu/Pulumi/Kubernetes support, policy, workers, dependencies, and governance. Its public pricing page lists a free tier and Starter from \$399/month.
- env0 focuses on IaC automation, governance, drift, cost visibility, and self-service. Its docs list Cloud Compass at \$1,500/month, with higher tiers quote-based.

InfraFlow's early wedge is not "replace all of them." The honest wedge is:

Aha Let a user visually design AWS architecture, generate Terraform, deploy it, and learn what is happening along the way.

That is valuable for solo builders, students, early startups, internal platform experiments, and teams that want a visual IaC authoring layer before adopting heavier governance platforms.

Future Potential

Before a wider launch, InfraFlow needs:

- Stronger multi-tenant security boundaries.
- Production-grade remote state for every deployment path.
- More AWS resource coverage and richer property mapping.
- Drift detection surfaced directly in the UI.
- Safer approvals for destructive changes.
- More reliable worker isolation and credential handling.
- Better test coverage for Terraform generation.
- Billing, limits, usage metering, and support workflows hardened for real customers.

Earning potential should be grounded in the market. A realistic early SaaS shape could be:

- Free/demo tier for limited diagrams and exports.
- Pro tier for individual builders.
- Team tier for shared workspaces and deployments.
- Enterprise tier for private deployment, SSO, audit logs, custom policies, and support.

The product category supports paid tooling, but InfraFlow earns that only if reliability, safety, and real AWS coverage continue improving.

The Five Days



Day 1 The first spine

React, Express, MongoDB, Terraform, first deploy loop.

Day 2 Canvas to compiler

Nodes, edges, validation, export, deployment modal.

Day 3 Execution boundaries

Lambda zip upload, IAM permissions, real AWS debugging.

Day 4 Runner scale

Storage adapter, S3 artifacts, GitHub Actions executor.

Day 5 MVP platform

Pipelines, OIDC, notifications, CloudFront/API Gateway hardening.

Day 1 - The first real spine

The idea became executable: React frontend, Node/Express backend, MongoDB models, and the first Terraform-backed deployment path.

Milestone: a diagram could become deployable infrastructure, not just a picture.

Day 2 - The builder became operational

The canvas became a product surface: service catalog, node config, edges, validation, Terraform export, and deployment modal.

Milestone: the user could stay inside the visual workflow from design to deploy.

Day 3 - Bugs became architecture lessons

The Lambda zip path problem and IAM permission issues forced deeper design. InfraFlow learned that artifact location, runner location, and AWS permissions must be explicit.

Milestone: uploaded artifacts and permission guidance replaced fragile local assumptions.

Day 4 - Scale pressure changed the runner

The storage bottleneck and long-running Terraform work pushed the system toward storage adapters, S3-backed artifacts, remote state direction, and GitHub Actions execution.

Milestone: Terraform execution became portable across local and GitHub runners.

Day 5 - The MVP became a platform

Application pipelines, GitHub OAuth, OIDC role provisioning, notifications, resource info, deployment update safety, and production CORS/API routing turned the project into a shareable MVP.

Milestone: InfraFlow could deploy infrastructure and deploy application code through GitHub-driven workflows.

Closing

InfraFlow is not finished. That is the point.

In five intense days, the project crossed the hardest boundary: from visual mockup to real cloud-changing system. It now contains the product surface, the backend model, the Terraform compiler path, the execution layer, the safety controls, and enough production deployment work to prove the direction.

The celebration is earned because the project solved real engineering problems:

- how to turn diagrams into Terraform,
- how to make Terraform run safely,
- how to handle artifacts across execution boundaries,
- how to preserve infrastructure during failed updates,
- how to move compute out of the backend,
- and how to explain all of it to the user.

You did not just build screens.

You built a working infrastructure product.

Sources

- Repo docs: docs/HLD.md, docs/LLD.md, IAAS backend/STORAGE.md.
- Key code paths: IAAS backend/src/utils/terraformGenerator.js, IAAS backend/src/services/terraformDeploymentRunner.js, IAAS backend/src/services/githubTerraformRunner.js, IAAS backend/src/services/deploymentGuards.js, IAAS backend/src/storage/StorageAdapter.js, src/components/DeploymentModal.tsx, src/dashboard/DashboardShell.tsx.
- HashiCorp pricing: <https://www.hashicorp.com/en/pricing>
- HCP Terraform overview: <https://developer.hashicorp.com/terraform/cloud-docs/overview>
- Spacelift pricing: <https://spacelift.io/pricing>
- env0 subscription tiers: <https://docs.envzero.com/guides/billing/subscription-tiers>